ARGO CD GITOPS › MY LEARNING NOTES › SESSION 04

Screenshots: Application Health Checks

Raw screenshots captured while watching this session (sync vs health status, 6 health states, HPA drift, ignoreDifferences, parent-child health, reconciliation timer), kept alongside the written notes (04-application-health-checks.md).

18 screenshots · captured 2026-09-12, 3:46 PM - 4:38 PM

Application Health Checks

Argo CD Application Health Checks evaluate whether your deployed Kubernetes resources are actually running and functioning correctly.

While Sync Status answers "Does the cluster match my Git repo?"

Health Status answers "Is the live code actually working?"

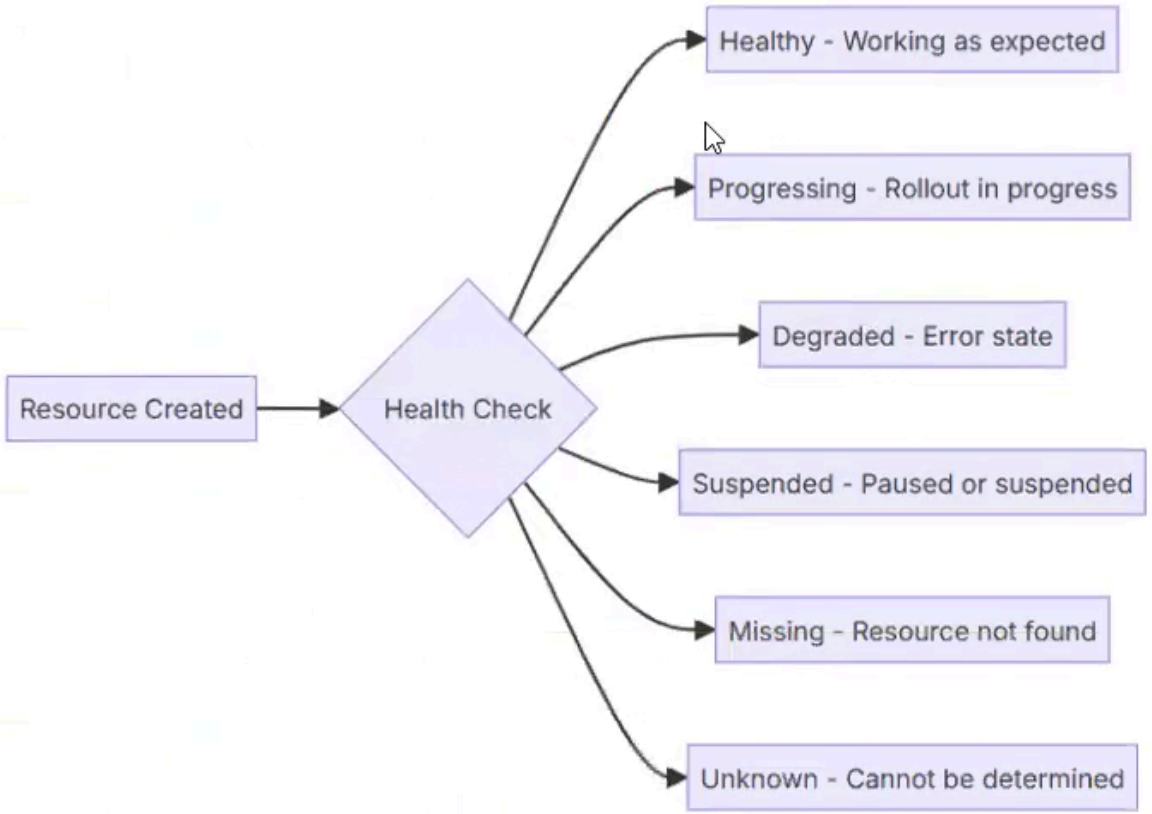
An application can be perfectly in-sync but totally degraded if its pods are crashing

The health status is separate from the sync status, and understanding the difference is critical for operating GitOps workflows.

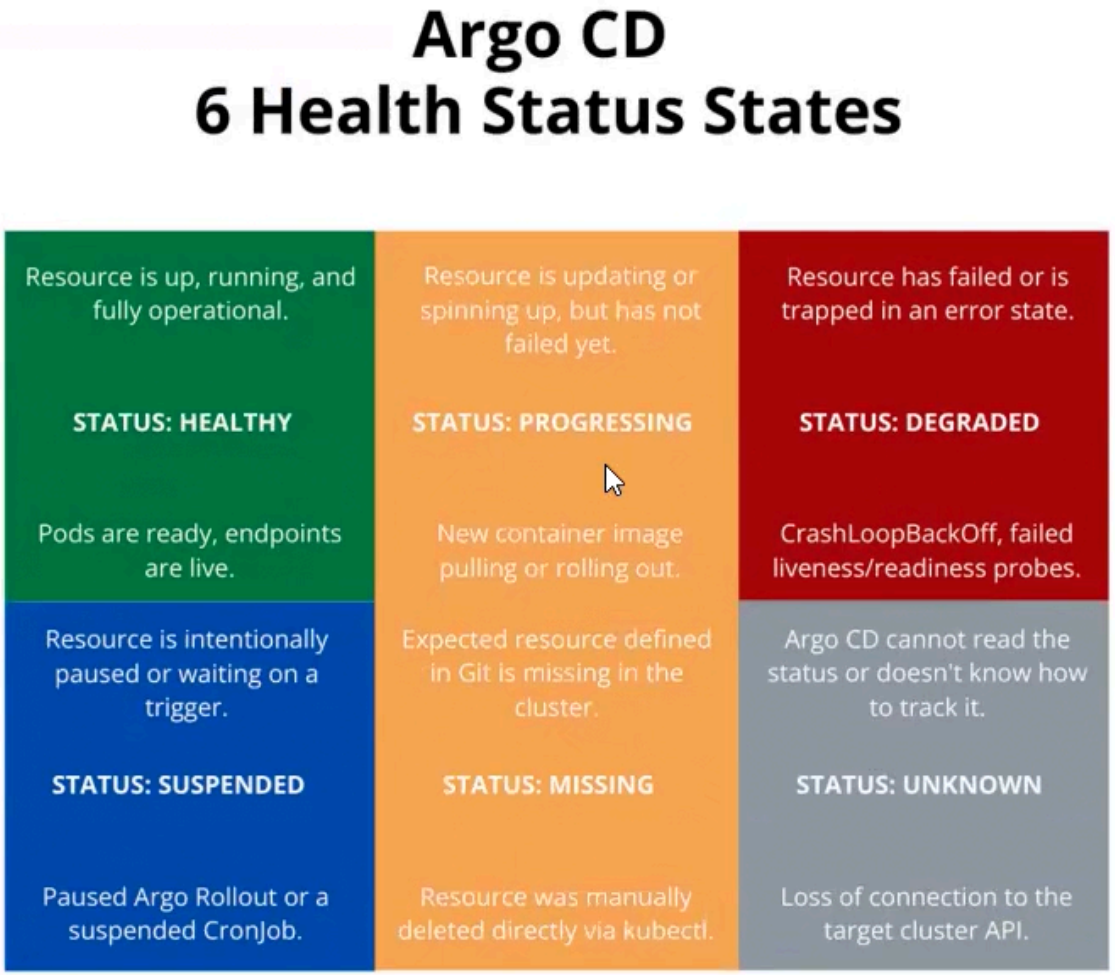
An application can be perfectly synced but unhealthy (all Pods are crashing), or out of sync but healthy (running an older version that works fine).

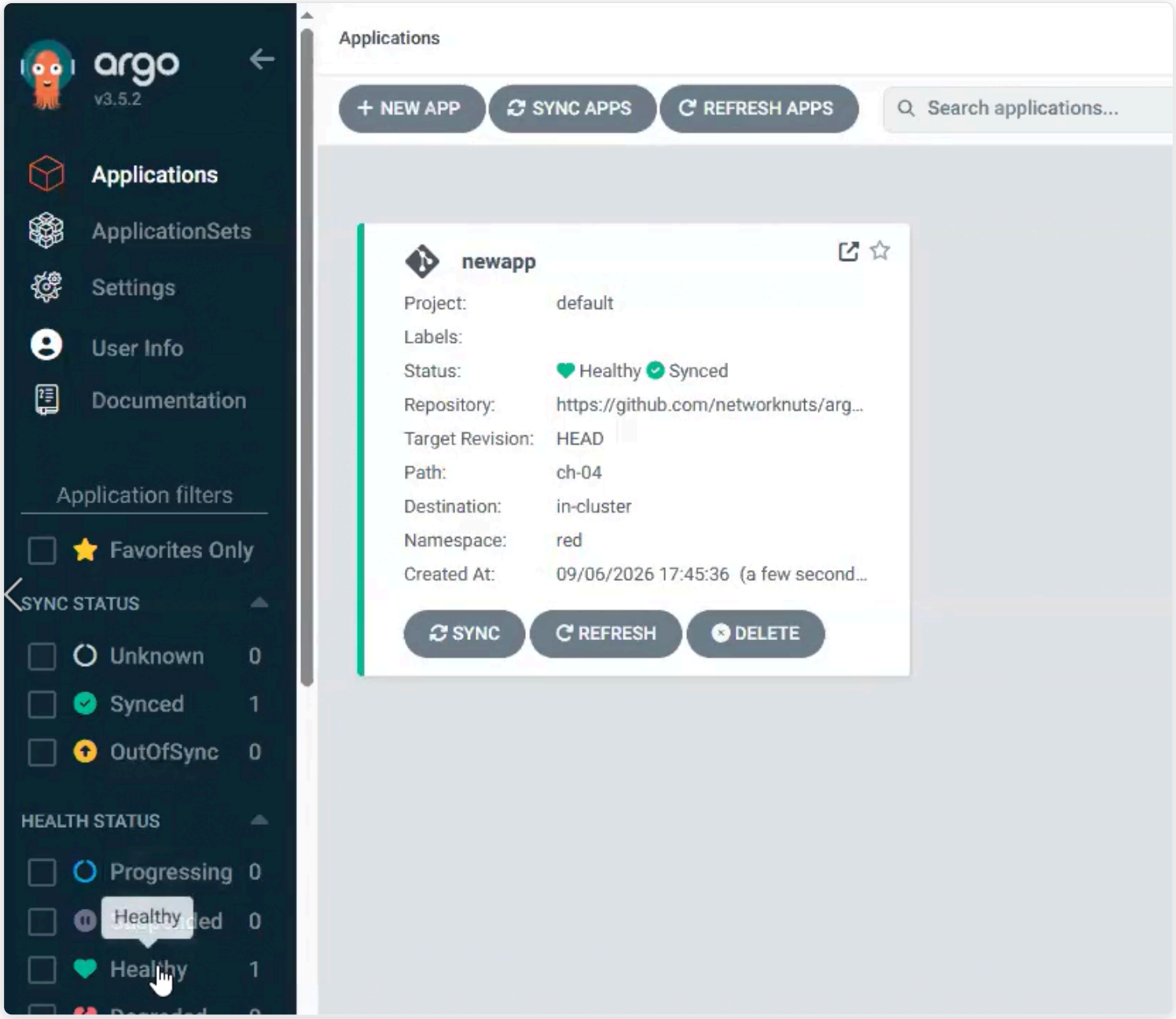
Application Health Checks

ArgoCD uses six health status values:



ArgoCD health status states




argo
v3.5.2
Applications
ApplicationSets
Settings
User Info
Documentation
Application filters
Favorites Only
SYNC STATUS
Unknown 0
Synced 1
OutOfSync 0
HEALTH STATUS
Progressing 0
Healthy
Healthy 1
Applications
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
newapp
Project: default
Labels:
Status: Healthy Synced
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: red
Created At: 09/06/2026 17:45:36 (a few second...
SYNC
REFRESH
DELETE

Application Health Checks

The lower a status appears in this list, the worse it is. If an application has multiple statuses, the lowest one in the list becomes the app's health.

- Missing + Degraded → Missing (because Missing is lower than Degraded).
- Degraded + Unknown → Unknown (Unknown is the lowest of all).
- Progressing + Degraded → Degraded.

This way, even if only one resource is in a very bad state, the application will reflect that worst-case status.

Argo CD provides built-in health assessment for several standard Kubernetes types, which is then surfaced to the overall Application health status as a whole.

The following checks are made for specific types of Kubernetes resources:

Deployment Health Check - ArgoCD evaluates Deployments by checking replica status:

Healthy when:

- status.updatedReplicas equals spec.replicas
- status.availableReplicas equals status.updatedReplicas
- No old ReplicaSets have running pods
- The Deployment generation matches the observed generation

Progressing when:

- Replicas are being updated (rolling update in progress)
- New pods are starting but not yet available
- The rollout has not exceeded progressDeadlineSeconds

Degraded when:

- The Deployment condition has reason ProgressDeadlineExceeded, which Kubernetes sets when the Deployment exceeds progressDeadlineSeconds (defaults to 600 seconds)

Application Health Checks

HorizontalPodAutoscaler Health Check

Healthy when:

- The HPA has an AbleToScale or ScalingLimited condition with status True

Progressing when:

- ArgoCD is waiting for autoscaling conditions

Degraded when:

- The HPA condition has AbleToScale with reason FailedGetScale or FailedUpdateScale
- The HPA condition has ScalingActive with reason FailedGetResourceMetric or InvalidSelector

```yaml
# argocd-application-for-hpa.yaml

##Step #1 - Create Deployment

---
#file start here

apiVersion: apps/v1
kind: Deployment
metadata:
  name: argolab
  namespace: production
  labels:
    app: argoapp
spec:
  # IMPORTANT: 'replicas' is completely removed.
  # This prevents GitOps drift detection from triggering on scale events.
  selector:
    matchLabels:
      app: argoapp
  template:
    metadata:
      labels:
        app: argoapp
    spec:
      containers:
      - name: boxone
        image: nginx:1.27
        ports:
        - containerPort: 80
        resources:
          limits:
            cpu: 200m
```

```yaml
kind: HorizontalPodAutoscaler
metadata:
  name: argolab-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: argolab
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
```

```
ignoreDifferences: #Ignore the dynamic replica count managed by the cluster's HPA controller
  - group: apps
    kind: Deployment
    jsonPointers:
      - /spec/replicas
```

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    - CreateNamespace=true
    - RespectIgnoreDifferences=true  #IMP:Tells Argo CD not to overwrite ignored fields during automated syncs
```

Application Health Checks

Ignoring Health Checks

Some resources don't need health monitoring. For example, you can exclude jobs that run once.

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: sample
  metadata:
    annotations:
      argocd.argoproj.io/ignore-healthcheck: "true"
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.34.0
        command: ["perl",  "-Mbignum=bpi", "-wle", "print bpi(2000)"]
      restartPolicy: Never
  backoffLimit: 4
```

Application Health Checks

Parent–Child Health in Argo CD

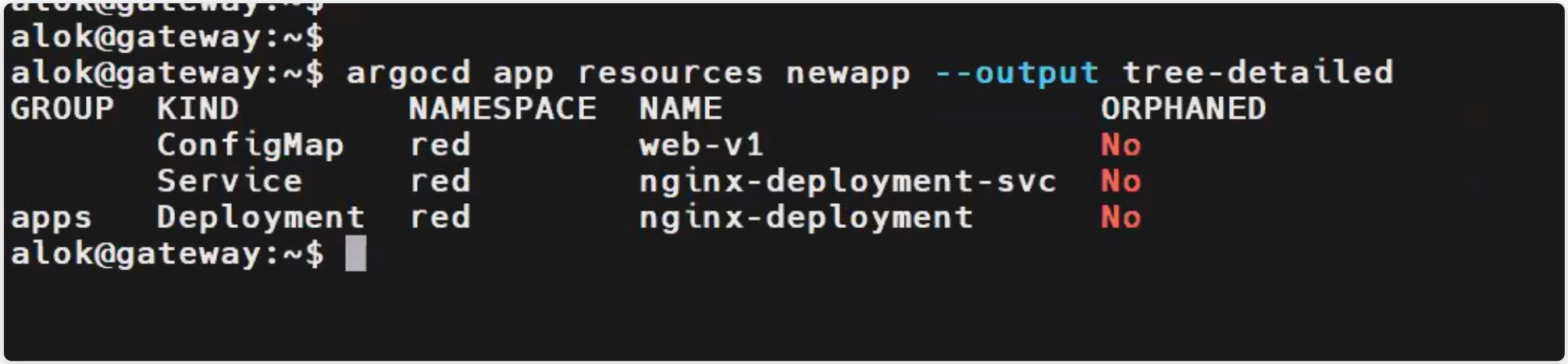
In Argo CD, health status is not automatically inherited from child resources. Each resource's health is calculated using only its own state

```
App: Healthy
└── Deployment: Healthy
    ├── ReplicaSet: Healthy
    │   └── Pod: Healthy
    └── ReplicaSet: Unhealthy
        └── Pod: Unhealthy
```

This means:

- A Deployment's health is decided based on its own conditions (replicas available, updated, etc.).
- The health of its Pods doesn't automatically affect the Deployment's health.
- A resource's health check may use information about its children only if the parent's status includes that info.

Not all child resources are relevant to the parent's health. For example, a Deployment might temporarily have one unhealthy ReplicaSet during a rollout, but that doesn't mean the Deployment itself is unhealthy — it might be progressing as expected.


alok@gateway:~$
alok@gateway:~$ argocd app resources newapp --output tree-detailed
GROUP KIND NAMESPACE NAME ORPHANED
ConfigMap red web-v1 No
Service red nginx-deployment-svc No
apps Deployment red nginx-deployment No
alok@gateway:~$

```
alok@gateway:~$
alok@gateway:~$ argocd app resources newapp --output tree=detailed
GROUP   KIND         NAMESPACE  NAME                                  ORPHANED  AGE  HEALTH   REASON
        ConfigMap    red        web-v1                                No        23h
        Service      red        nginx-deployment-svc                  No        24h  Healthy
apps    Deployment   red        nginx-deployment                      No        24h  Healthy
└─apps  ReplicaSet   red        nginx-deployment-77bc6bd484           No        24h  Healthy
  ├─    Pod          red        nginx-deployment-77bc6bd484-25lwg     No        24h  Healthy
  ├─    Pod          red        nginx-deployment-77bc6bd484-5s598     No        24h  Healthy
  ├─    Pod          red        nginx-deployment-77bc6bd484-rp22m     No        24h  Healthy
```

```
  ├─     Pod          red         nginx-deployment-77bc6bd484-rp22m  No         24h  Healthy
  └─     Pod          red         nginx-deployment-77bc6bd484-x7wxt  No         24h  Healthy
alok@gateway:~$ argocd app resources argolab-production --output tree=detailed
GROUP        KIND                     NAMESPACE   NAME                       ORPHANED  AGE  HEALTH   REASON
apps         Deployment               production  argolab                    No        11m  Healthy
└─apps       ReplicaSet               production  argolab-7b9867486          No        11m  Healthy
  ├─         Pod                      production  argolab-7b9867486-rt285    No        10m  Healthy
  └─         Pod                      production  argolab-7b9867486-w75qx    No        11m  Healthy
autoscaling  HorizontalPodAutoscaler  production  argolab-hpa                No        11m  Healthy  recommended size match
es current size
alok@gateway:~$
```

Application Health Check - Timers

Global Reconciliation Timer

Argo CD evaluates the health of all resources inside an application when it runs its periodic background check. By default, this happens every 180 seconds (3 minutes).

If you want Argo CD to re-evaluate and refresh the health of your resources faster or slower, change the **timeout.reconciliation** value in the argocd-cm ConfigMap.

#kubectl -n argocd edit cm argocd-cm

```
data:
  timeout.reconciliation: "60s"
```

##save and exit

#kubectl -n argocd rollout restart statefulset argocd-application-controller

```
#kubectl -n argocd rollout restart statefulset argocd-application-controller
```